

Algorithms and Data Structures I.

Practice 4

Gábor Pusztai

Department of Algorithms And Their Applications
Faculty of Informatics
ELTE - Eötvös Loránd University, Budapest

March 8, 2024

Content in this Practice I

- 1 Review last week -
Divide and Conquer algorithms
 - Quicksort - Example
- 2 Data Structure - Stack
 - Stack
 - Stack - Type
 - Stack - Operations
 - Stack - Example
- 3 Problems for stacks
 - Correct parenthesis syntax
 - Counter solution
 - Denoting the parenthesis
 - Reversed ordering of names
- 4 Reversed Polish Notation

Content in this Practice II

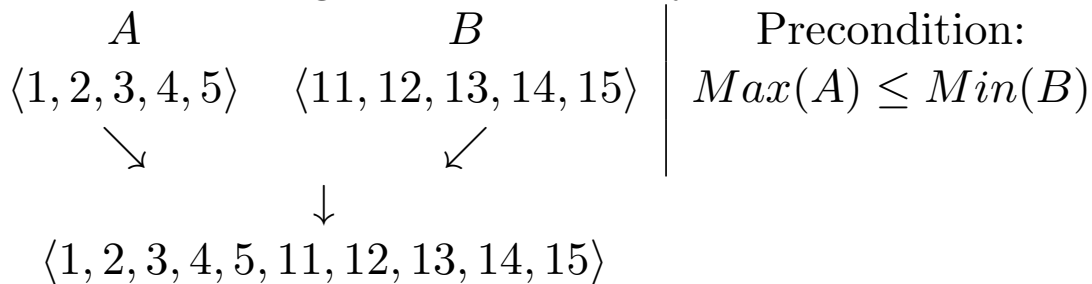
- History
- Reversed Polish Notation
- Conversion from infix to postfix
- Evaluation from Polish notation

Review last week - Divide and Conquer algorithms

Quicksort - Main idea

- Main Idea:

If the left and the right arrays are sorted and every element in the left array is smaller than any element in the right, I can put them together without any extra process.



Quicksort - Example

Array:	$\langle 12, 31, 23, 20, 14, 55, 11 \rangle$
Choosing the <i>PIVOT</i> item:	$\langle 12, 31, 23, 20, \underline{14}, 55, 11 \rangle$
Swap with the last element:	$\langle 12, 31, 23, 20, 11, 55, \underline{14} \rangle$
Look for $A[i] > \textit{PIVOT}$:	$\langle 12, \underline{31}, 23, 20, 11, 55, \underline{14} \rangle$
Set $j := i + 1$:	$\langle 12, \underline{31}, \underline{23}, 20, 11, 55, \underline{14} \rangle$
Look for $A[j] < \textit{PIVOT}$:	$\langle 12, \underline{31}, 23, 20, \underline{11}, 55, \underline{14} \rangle$
SWAP $\underline{A[i]}$ and $\underline{A[j]}$:	$\langle 12, \underline{11}, 23, 20, \underline{31}, 55, \underline{14} \rangle$
Set $i := i + 1$:	$\langle 12, 11, \underline{23}, 20, \underline{31}, 55, \underline{14} \rangle$
Continue $A[j] < \textit{PIVOT}$:	$\langle 12, 11, \underline{23}, 20, \underline{31}, 55, \underline{14} \rangle$
j reaches the <i>PIVOT</i> :	$\langle 12, 11, \underline{23}, 20, 31, 55, \underline{14} \rangle$
SWAP $\underline{A[i]}$ with the <i>PIVOT</i> :	$\langle 12, 11, \underline{14}, 20, 31, 55, \underline{23} \rangle$
$\langle A \text{ (small)} \rangle \textit{PIVOT} \langle B \text{ (big)} \rangle$:	$\langle 12, 11 \rangle \underline{14} \langle 20, 31, 55, 23 \rangle$

Data Structure - Stack

Stack

Illustration problem: You just moved to a new room with all your stuff still in boxes. You have limited space, how do you save some space?

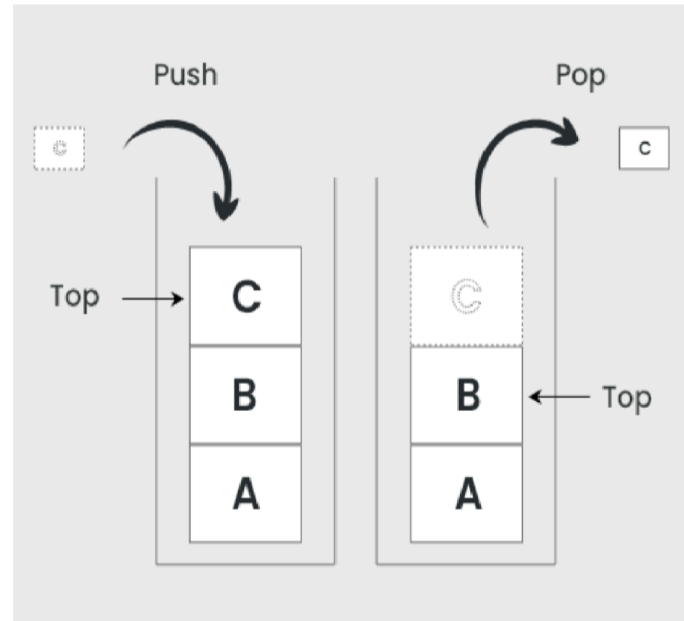
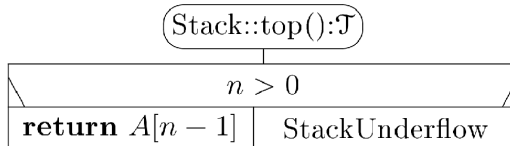
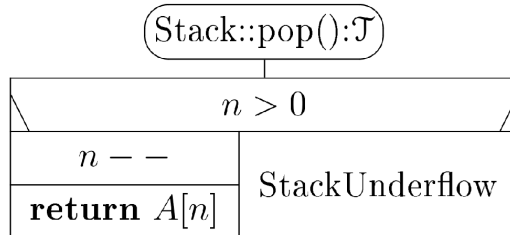
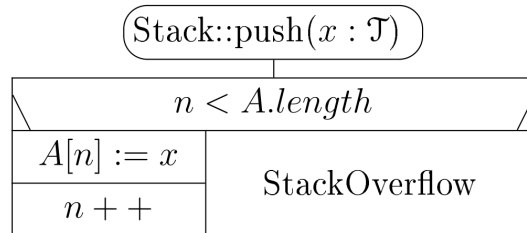
You put the boxes on top of each other.

- Bring in the new box into the room.
- Put it on top of the previous box.

Stack

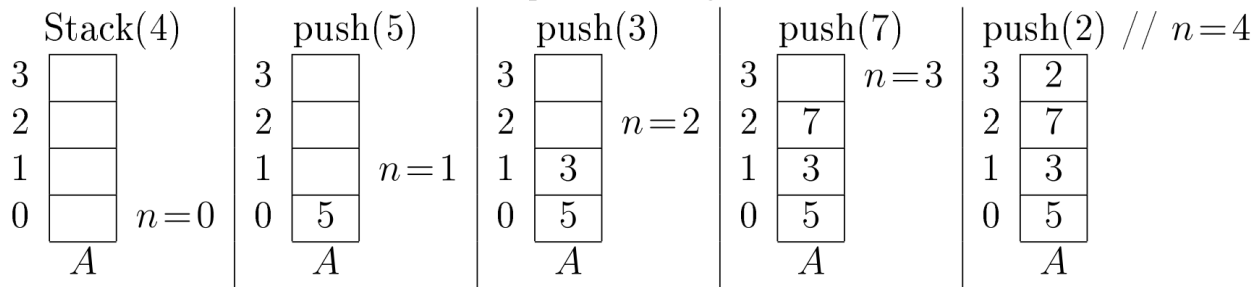
Stack
$-A : \mathcal{T}[]$ // $\mathcal{T}$ is some known type ; $A.length$ is the max. size of the stack $-n : \mathbb{N}$ // $n \in 0..A.length$ is the actual size of the stack
$+ \text{Stack}(m : \mathbb{N}) \{A := \text{new } \mathcal{T}[m] ; n := 0\}$ // create an empty stack $+ \text{push}(x : \mathcal{T})$ // push x onto the top of the stack $+ \text{pop}() : \mathcal{T}$ // remove and return the top element of the stack $+ \text{top}() : \mathcal{T}$ // return the top element of the stack $+ \text{isFull}() : \mathbb{B} \{\text{return } n = A.length\}$ $+ \text{isEmpty}() : \mathbb{B} \{\text{return } n = 0\}$ $+ \text{setEmpty}() \{n := 0\}$ // reinitialize the stack $+ \sim \text{Stack}() \{ \text{delete } A \}$

Stack



Stack

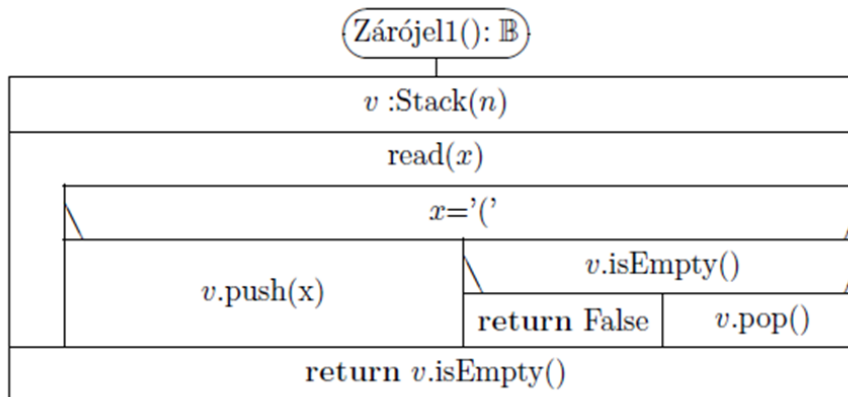
Some operations of a Stack



Problems for stacks

Correct parenthesis syntax

- Problem: Given a series of parentheses as input. How can we check if the syntax is 'correct' or not?
- By correct we mean the natural interpretation: Every *opening* parenthesis can be matched with a *closing* one occurring later in the sequence.
- Correct: ((() ()))
- Not correct: (() OR ()))



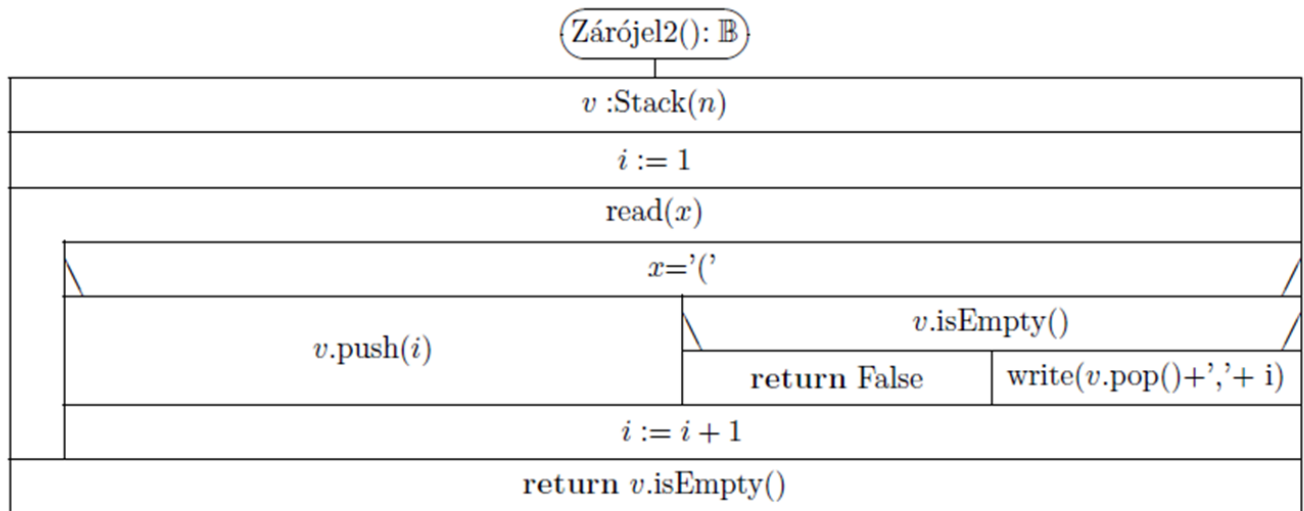
Problems for stacks - Counter solution

- We don't need a stack for this. Just a counter.
- We start from 0.
- Finding an opening parenthesis means increasing by one and finding a closing means subtracting one.
- The expression is correct if during the algorithm the counter doesn't turn negative, and by the end, it becomes 0.

	(	(	)	(	)	)	(	(	(	)	(	)	)	)
0	1	2	1	2	1	0	1	2	3	2	3	2	1	0

Problems for stacks - Denoting the parenthesis

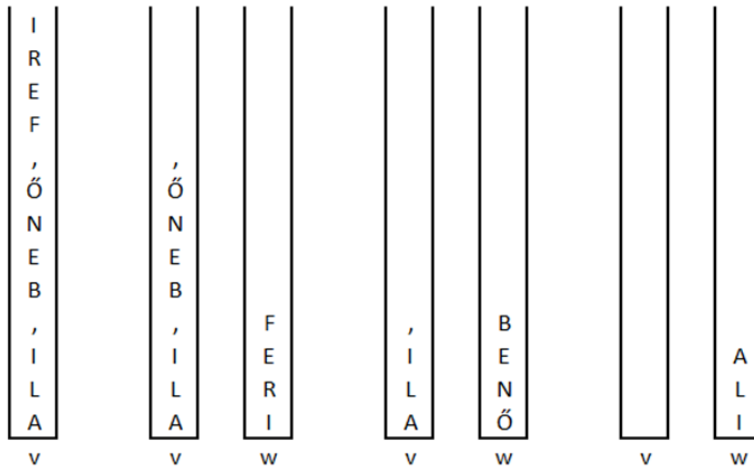
- Modify the stack algorithm such that we print the 'id' of the parenthesis pair.



Problems for stacks - Reversed ordering of names

- Given a single string representing multiple names separated by commas as input, which we can only read character by character.
- We wish to reverse the order the names appear.
- Example:
 - Input: Ali, Benő, Feri, Juci, Tomi
 - Output: Tomi, Juci, Feri, Benő, Ali
- For the solution we can use two stacks

Problems for stacks - Reversed ordering of names

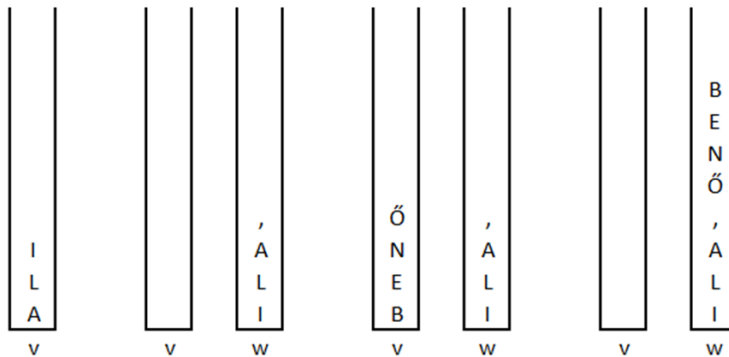


Kimenet:

F E R I , B E N Ö , A L I

- First solution:
- Read the whole thing into a single stack.
- Starting from the top, we put each character into the second one until the comma, and then flush the second stack.
- Repeat until the first stack is not empty.

Problems for stacks - Reversed ordering of names 2



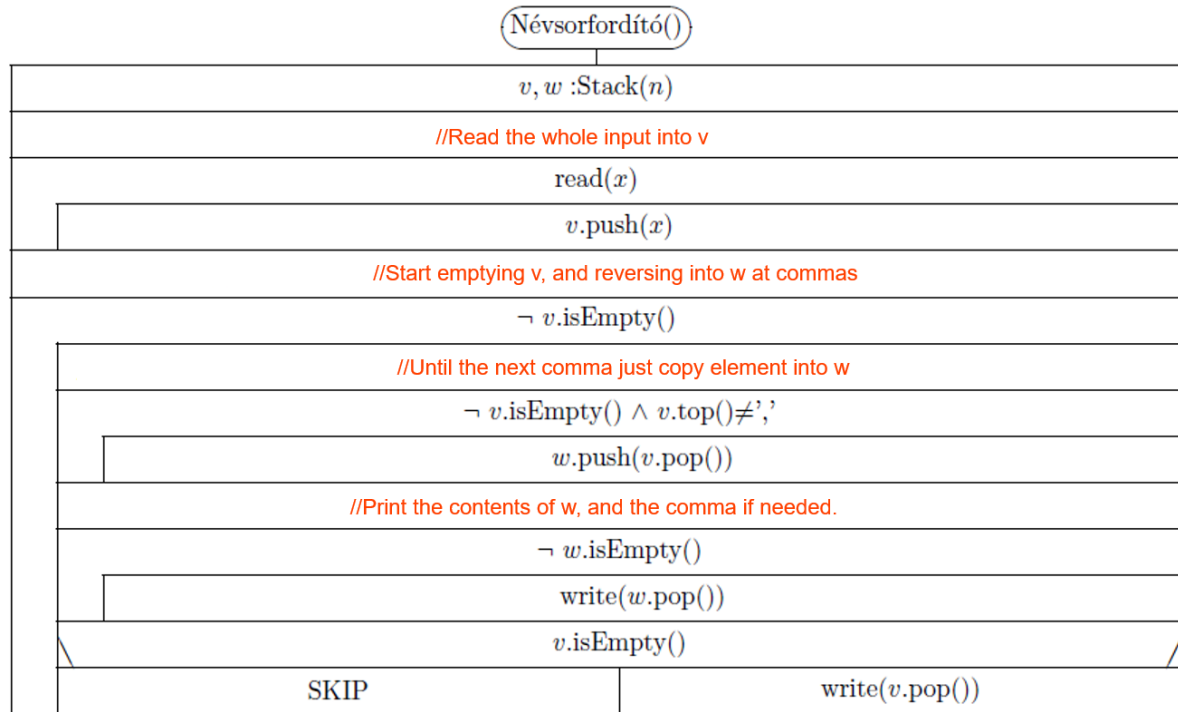
Kimenet:

B E N Ő , A L I

- Second solution:
- Read the input until a comma into the first stack.
- After reaching the first comma flush the first stack into the second one.
- Repeat until we reach the end of the input, and then flush the second stack.

Problems for stacks - Reversed ordering of names

- Structogram of the first solution



Reversed Polish Notation

Reversed Polish Notation - History

Jan Łukasiewicz



- Originally Jan Łukasiewicz, a Polish mathematician proposed an alternative, the prefix notation (1920):
- Followed by Charles Leonard Hamblin from Australia computer scientist who proposed the *postfix* version (1950).
- However, the Polish notation term stuck with us regardless, so even the postfix is called Polish notation (or Reversed Polish Notation).

Reversed Polish Notation

- A mathematical notation for evaluating algebraic expressions efficiently (by a compiler).
- The usual '*human*' notation is if, add 3 and 4, it is called **infix** we write
 $3 + 4$
where the **operator** (+) is between the **operands** (3, 4)
- But why don't we write it, where the operators follow their operands, it is called **postfix**
 $3\ 4\ +$
- For multiple operations, instead of
 $3 + 4 * 5 \rightarrow 3\ 4\ 5\ * \ +$

Reversed Polish Notation

- What is the correct order of operations?
- Mathematical expressions have a certain precedence (ordering)
 - $a+b*c \rightarrow b*c \rightarrow a+(b*c)$
 - $a*b+c \rightarrow a*b \rightarrow (a*b)+c$
- But we can override this precedence by using parentheses.
 - $a/(b-c)$
 - $(a*(b-c))/((x-y)*(x-z))$
- Some operations are on the same level of precedence.

Reversed Polish Notation

- Some examples infix $\rightarrow$ postfix
 - $a+b \rightarrow ab+$
 - $a*b+c \rightarrow ab*c+$
 - $a+b*c \rightarrow abc*+$
 - $a/b*c \rightarrow ab/c*$
 - $(a-b)/(c+d) \rightarrow ab-cd+ /$
 - Single operand operator (neg. sign)

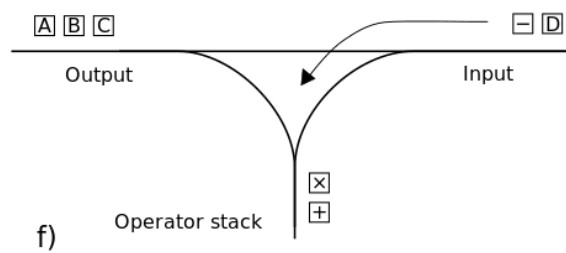
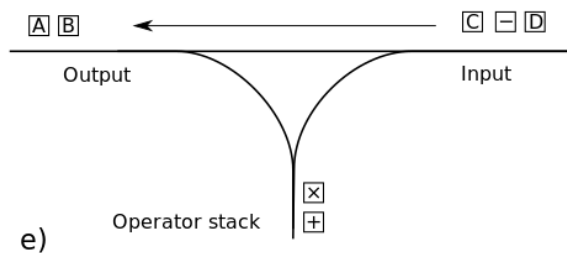
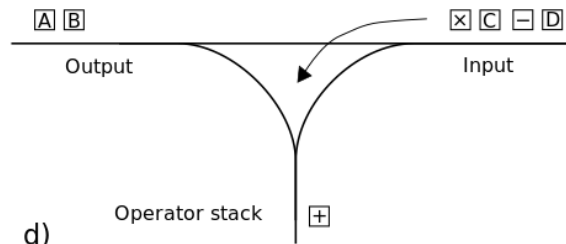
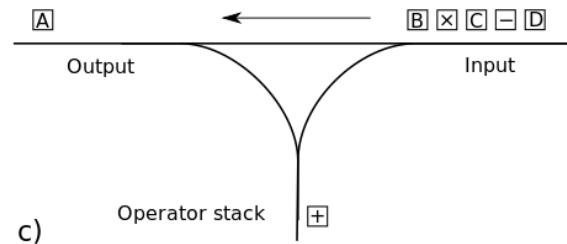
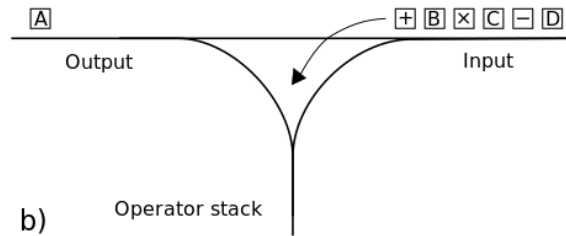
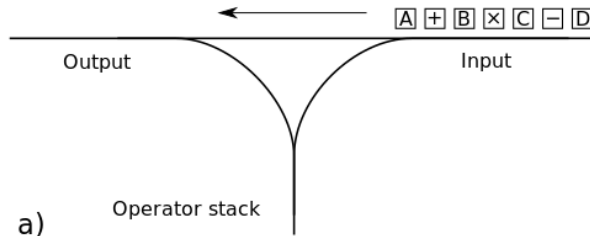
Reversed Polish Notation

- Properties of the postfix notation
 - NO PARENTHESES
 - The order of operands remain the same as in the infix notation
 - Operators directly follow their operands, meaning if an operator appears, it definitely means that its inputs are the ones before it.
 - These inputs can be either simple operands, or a result of another postfix notation.

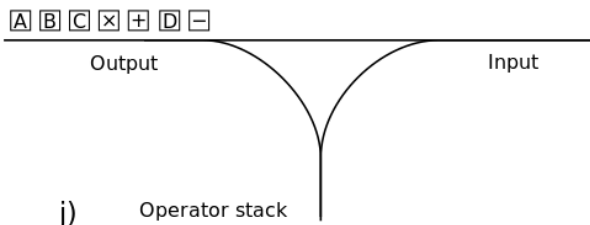
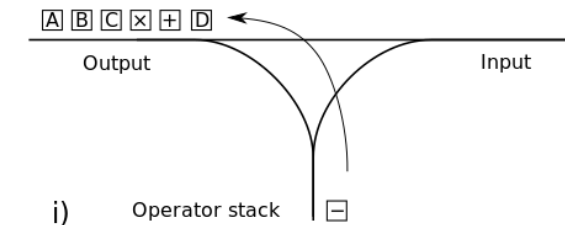
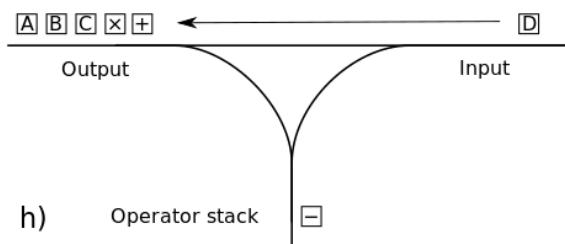
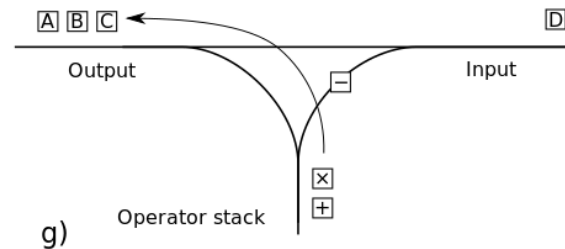
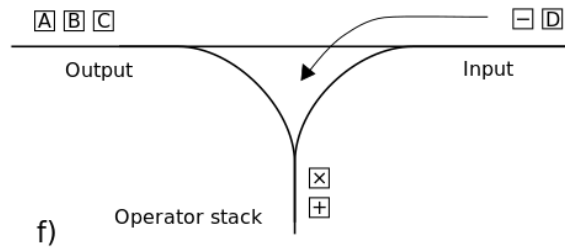
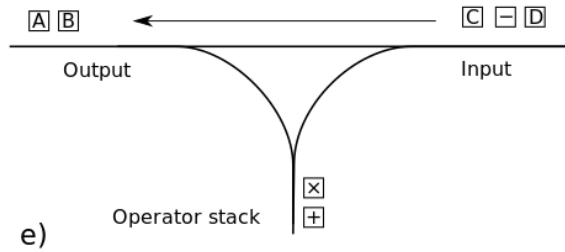
Conversion from infix to postfix

- Dijkstra: Shunting yard algorithm
 - A method for parsing arithmetical or logical expressions, or a combination of both, specified in infix notation. It can produce a postfix notation string.
 - For the conversion there are two text variables (strings), the *input* and the *output*.
 - There is also a **stack** that holds operators not yet added to the output queue.
 - To convert, the program reads each symbol in order and does something based on that symbol.

Graphical illustration of algorithm



Graphical illustration of algorithm



Conversion from infix to postfix

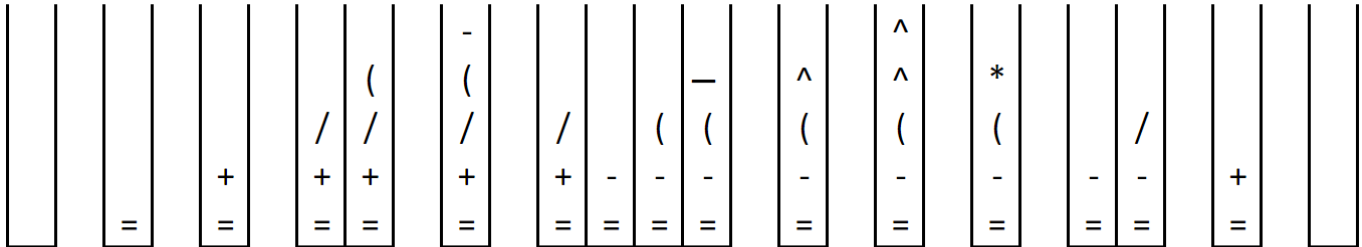
Original expression: $x = b + 2 / (a - c) - (-f \wedge b \wedge 2 * g) / z + w$

Precedence ranking:

$\text{prec}(=) < \text{prec}(+) = \text{prec}(-) < \text{prec}(\wedge) < \text{prec}(/) < \text{prec}(\wedge) < \text{prec}(-)$

Contents of the stack: For each character whatever is currently in the stack

x = b + 2 / (a - c) - (- f ^ b ^ 2 * g) / z + w



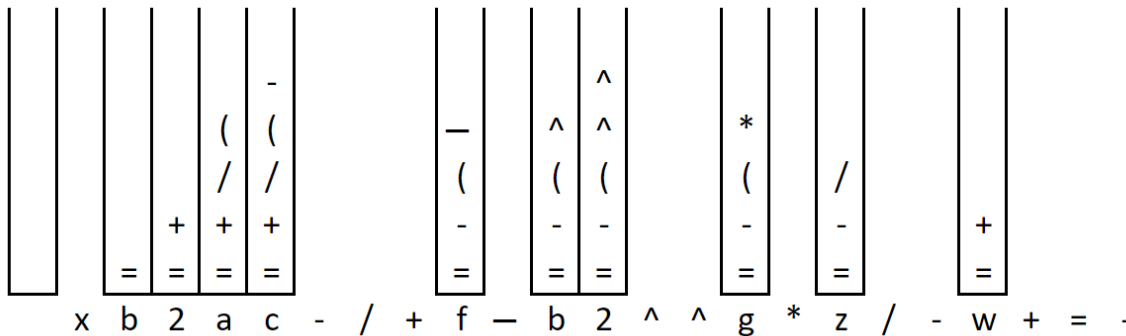
Resulting polish notation

x b 2 a c - / + f - b 2 ^ ^ g * z / - w + =

Conversion from infix to postfix

More compact if we just write the contents of the stack for every operand we write out.

Original expression: $x = b + 2 / (a - c) - (-f \wedge b \wedge 2 * g) / z + w$



Conversion from infix to postfix

Polish Notation (**n:N**)

V : Stack(n)				
read(x)				
$x \in \{ \text{operands} \}$	$x = '('$	$x = ')'$	$x \in \{ \text{LR operators} \}$	$x \in \{ \text{RL operators} \}$
Write(x)	V.push(x)	$V.\text{top}() \neq '('$	$\neg V.\text{isEmpty}() \wedge V.\text{top}() \neq '(' \wedge \text{prec}(V.\text{top}()) \geq \text{prec}(x)$	$\neg V.\text{isEmpty}() \wedge V.\text{top}() \neq '(' \wedge \text{prec}(V.\text{top}()) > \text{prec}(x)$
		Write(V.pop())		
		V.pop()	Write(V.pop())	Write(V.pop())
			V.push(x)	V.push(x)
$\neg V.\text{isEmpty}()$				
Write(V.pop())				

- LR operators: Set of operators which has precedence from left to right (+ - * / ...)
- RL operators: Set of operators which has precedence from right to left (^ := ...)

Evaluation from Polish notation

- Let us have the expression: $(a + b) / c - d^2$
- Which has the polish notation: $a\ b\ +\ c\ /\ d\ 2\ ^\ -$
- Let the variables be $a=12$, $b=8$, $c=2$, $d=3$. Evaluate the expression!
- Method:
 - Read the next character in Polish notation.
 - If it's an operand, push into the stack.
 - If it's an operator, pop the top operands needed, evaluate the expression, and push back the result.
 - Repeat until the end of the polish notation. The result is the element in the stack.

Evaluation from Polish notation

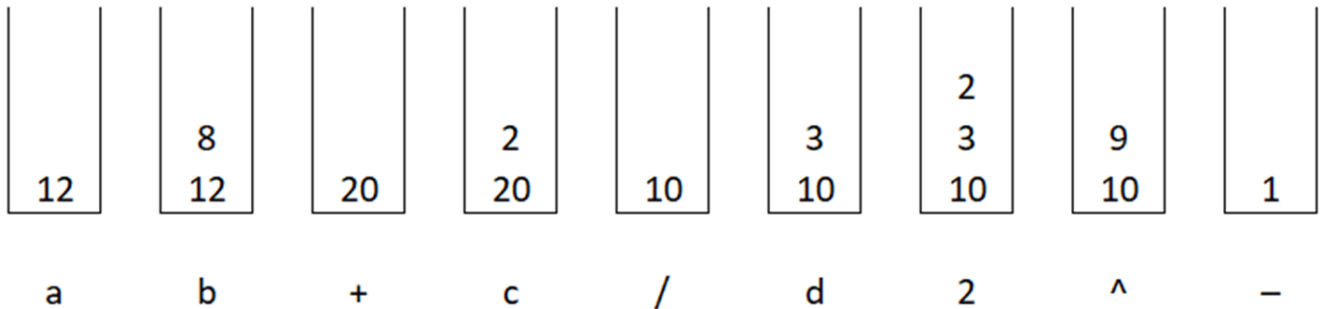
$a \ b \ + \ c \ / \ d \ 2 \ ^ \ -$

$a=12$

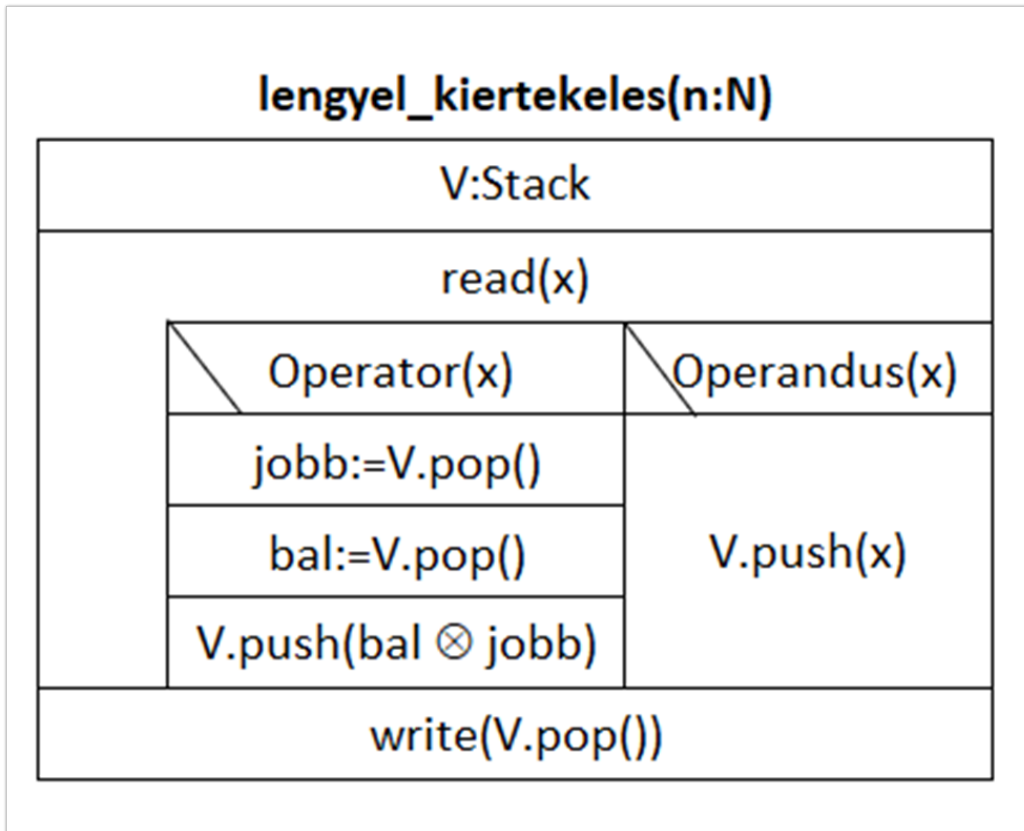
$b=8$

$c=2$

$d=3$



Evaluation from Polish notation



Ending

Thank you for your attention!